

DIGITAL IC
DESIGN CAMP

LEARN | DESIGN | BUILD

FROM
IDEAS TO
REAL CIRCUITS

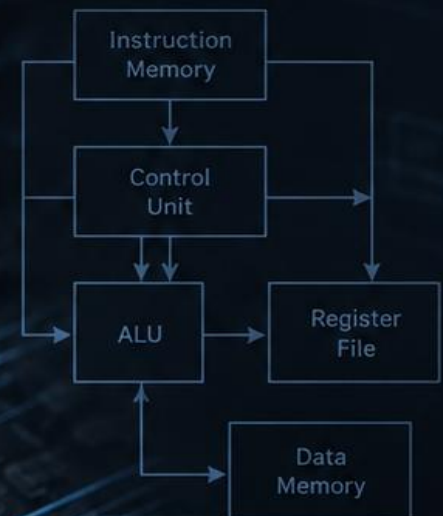
FINAL PROJECT

MINI PROCESSOR

SINGLE-CYCLE CPU

DIGITAL IC DESIGN & VERIFICATION

SMALL
ARCHITECTURE
BIG
POTENTIAL



CLK
PC
INSTR
RegWrite
Zero



Prepared By

Ahmed Emad

Electrical Engineering
Electronics and Communication Systems

HARDWARE
CREATES
POSSIBILITIES

Table of Contents

Contents

1. Project Overview	3
2. System Architecture	3
2.1 Program Counter:	4
2.2 Instruction Memory:	5
2.3 Register File	5
3. Instruction Set Architecture (ISA)	6
3.1 R-Type Instructions	6
3.2 I-Type Instructions	7
3.3 Branch and Jump Instructions	7
3.4 System Instructions	9
4. Module Implementation	9
4.1 Program Counter (PC)	9
4.2 Instruction Memory (IMEM)	10
4.3 Register File	12
4.4 Arithmetic Logic Unit (ALU)	14
4.5 Control Unit	16
4.6 Top-Level Mini Processor	18
4.7 Testbench	19
5. Functional Requirements	20
5.1 Instruction Execution	20
5.2 Program Counter Control	20
5.3 Register and ALU Operations	20
5.4 Verification	20
6. Verification	21
6.1 Main Instruction Test	21
6.2 BEQ Taken Test	22
6.3 BNE Test	24
6.4 HALT Verification	26
7. Future Improvements	27

1. Project Overview

The **Mini Processor** is the final project of the Digital IC Design Camp. The project aims to integrate the previously designed and verified **ALU** and **Register File** into a complete programmable processor, together with two new blocks: the **Program Counter (PC)** and **Instruction Memory (IMEM)**. A **Control Unit** is also used to control the operation of the different blocks according to the current instruction.

The processor follows a **Single-Cycle CPU architecture**, where the current instruction is fetched, decoded, and executed during the same clock cycle. At the active clock edge, the required register write and PC update are committed.

The processor uses a custom **16-bit Instruction Set Architecture (ISA)**. The two most significant bits, [15:14], determine the instruction class, allowing the processor to distinguish between register-based operations, immediate operations, control-flow instructions, and system instructions.

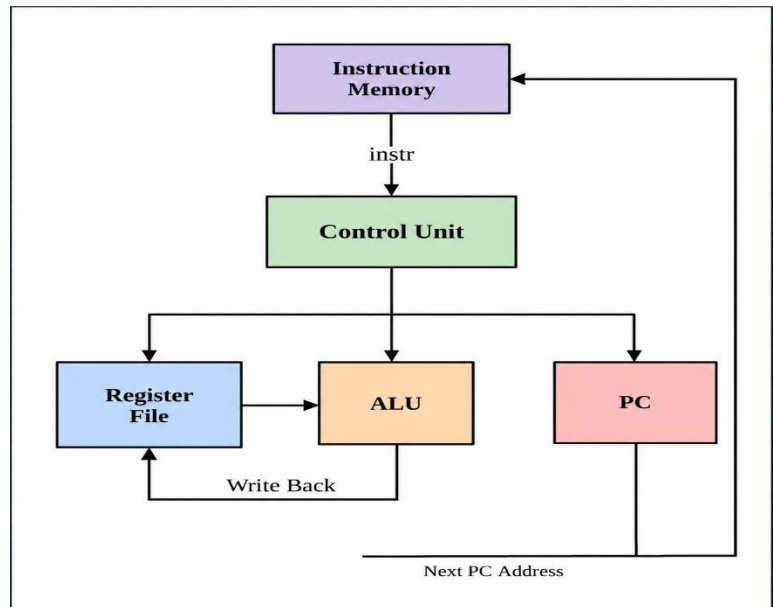
The implemented processor contains four 8-bit registers, a 6-bit Program Counter, a 64-location instruction memory, and an 8-bit ALU. This architecture provides the basic functionality required to execute arithmetic and logical operations, load immediate values, change program flow using branches and jumps, and stop execution using the HALT instruction.

2. System Architecture

The Mini Processor is organized as a small single-cycle CPU in which several previously designed digital blocks are integrated together to form one complete processing data path. The main architecture consists of five blocks: the **Program Counter (PC)**, **Instruction Memory (IMEM)**, **Register File**, **ALU**, and **Control Unit**.

The basic relationship between these blocks can be understood from the execution of any instruction. The **PC** determines the address of the current instruction, and this address is supplied to the **Instruction Memory**. The Instruction Memory then provides the corresponding 16-bit instruction. This instruction is interpreted by the **Control Unit**, which generates the control signals required to operate the data path. The **Register File** provides the required source operands, while the **ALU** performs the required arithmetic or logical operation. Depending on the instruction, the resulting data may be written back to the Register File, while the Control Unit also determines the next value of the PC.

The overall data flow can therefore be represented as:



The important point is that the **Instruction Memory does not decode the instruction**. It only receives a 6-bit address and returns the corresponding 16-bit instruction. The fields such as rs1, rs2, rd, and immediate are extracted from the instruction by the surrounding top-level wiring and control logic.

2.1 Program Counter:

The Program Counter is responsible for identifying the address of the instruction currently being executed. Since the Instruction Memory contains 64 locations, a **6-bit PC** is sufficient to address all instructions.

The PC can receive four different types of updates through the pc_src control signal:

pc_src	Operation
00	PC + 1
01	Jump target
10	Branch target
11	Hold current PC

This allows the processor to normally execute instructions sequentially, while also supporting jumps, conditional branches, and the HALT operation.

2.2 Instruction Memory:

The Instruction Memory stores the program that will be executed by the processor. It is implemented as a **64 × 16-bit read-only memory**, matching the 6-bit address generated by the PC.

Conceptually, its operation is very simple:

PC Address —→ Instruction Memory —→ 16-bit Instruction

For example, if: PC = 0 → the memory returns the instruction stored at location 0. In our test program, this instruction is: 0100000001010000 which represents: **LOADI R0, 5**

The memory itself does not need to understand that meaning; it simply returns the stored 16-bit word.

2.3 Register File

The Register File provides the processor with its internal data storage. It contains four registers, R0 through R3, with each register storing **8 bits**.

Because an instruction can require two source operands, the Register File provides two read paths. It also contains one synchronous write path for storing the result of an instruction. The project specification requires the existing register-file structure and write logic to be preserved while extending it to support the second read port.

This is particularly important for an instruction such as: ADD R2, R0, R1

because the processor must obtain both: R0 → ALU input A , R1 → ALU input B

before producing the result that will eventually be written into R2.

3. Instruction Set Architecture (ISA)

The processor uses a custom **16-bit Instruction Set Architecture (ISA)**. Each instruction is represented by a 16-bit word, and the instruction format is divided into different fields depending on the instruction type. The two most significant bits, [15:14], are used to identify the instruction class.

The implemented ISA is divided into four main instruction classes:

[15:14]	Instruction Type	Purpose
00	R-Type	Arithmetic and logical operations
01	I-Type	Load immediate value
10	Branch / Jump	Change program flow
11	System	HALT and NOP

The instruction format was designed so that the processor can determine the required operation and operands directly from the 16-bit instruction.

3.1 R-Type Instructions

R-Type instructions are used for operations where the operands are stored in the Register File. Their format is:

[15:14] [13:10] [9:8] [7:6] [5:4] [3:0]
 00 ALU Op rd rs1 rs2 Reserved

The fields represent:

- [15:14] = 00 → identifies an R-Type instruction.
- [13:10] → ALU operation code.
- [9:8] → destination register rd.
- [7:6] → first source register rs1.
- [5:4] → second source register rs2.
- [3:0] → reserved bits and set to zero.

For example: ADD R2, R0, R1

means that the processor reads R0 and R1, sends their values to the ALU, performs the ADD operation, and writes the result into R2.

In our implementation, the ALU operation field is directly connected to the ALU opcode, allowing the same operation table from the previous ALU assignment to be reused.

3.2 I-Type Instructions

I-Type instructions are used to load an immediate value directly into a register. The implemented I-Type instruction is **LOADI**.

Its format is:

[15:14]	[13:12]	[11:4]	[3:0]
01	rd	Immediate	Reserved

The immediate field is 8 bits and is **zero-extended** when used by the processor.

For example: `LOADI R0, 5` → is represented by: `0100000001010000`

The processor decodes this instruction as:

Instruction Type = I-Type

Destination = R0

Immediate = 5

The immediate value is then selected as the write-back data and stored in R0 at the active clock edge.

3.3 Branch and Jump Instructions

The 10 instruction class is used for changing the normal sequential execution of the program. It supports both **JMP** and **BEQ**.

JMP

When: `instr[15:14] = 10` , `instr[13] = 0`

the instruction is interpreted as an unconditional jump.

The target address is stored in: `instr[5:0]` → Therefore, instead of the normal: `(PC + 1)` the PC loads the specified target address.

Conceptually: `JMP target` → PC → target

BEQ: the instruction is interpreted as a **Branch if Equal** instruction

When: $\text{instr}[15:14] = 10$, $\text{instr}[13] = 1$

The two registers are specified by:

$\text{rs1} = \text{instr}[12:11]$, $\text{rs2} = \text{instr}[10:9]$ and the branch target is: $\text{target} = \text{instr}[5:0]$

To determine whether the registers are equal, the processor uses the ALU to perform: $R1 - R2$

If the result is zero, the zero flag becomes 1, indicating that the two operands are equal. The Control Unit then selects the branch target for the PC. If the result is not zero, the branch is not taken, and the PC continues with $\text{PC} + 1$.

This allows the existing ALU subtraction and zero detection logic to be reused instead of introducing another comparison block.

BNE — Branch if Not Equal

The **BNE (Branch if Not Equal)** instruction was implemented as an extension to the original branch instruction set. It allows the processor to branch to a target address when the values stored in two registers are different.

The BNE instruction reuses the same data path used by BEQ. The ALU performs a subtraction between the two source registers: $R1 - R2$

The zero flag is then used to determine whether the two operands are equal:

$\text{zero} = 1 \rightarrow R1 = R2 \rightarrow \text{BNE not taken}$, $\text{zero} = 0 \rightarrow R1 \neq R2 \rightarrow \text{BNE taken}$

To distinguish BNE from BEQ, an additional instruction bit was used in the branch encoding while keeping the existing register fields and 6-bit target address unchanged. In our implementation:

$\text{instr}[15:14] = 10$, $\text{instr}[13] = 1$, $\text{instr}[8] = 0 \rightarrow \text{BEQ}$, $\text{instr}[8] = 1 \rightarrow \text{BNE}$

The remaining branch fields are preserved:

$\text{instr}[12:11] \rightarrow \text{rs1}$, $\text{instr}[10:9] \rightarrow \text{rs2}$, $\text{instr}[5:0] \rightarrow \text{target}$

This implementation does not require any additional datapath hardware because the existing **ALU subtraction operation and zero flag** are sufficient to perform the comparison.

Final_project_Mini_cpu_dd_camp.pdfPDF

Example

For: BNE R1, R2, 7

if: $R1 = 7$, $R2 = 5 \rightarrow$ the ALU performs: $7 - 5 = 2$

Therefore: $\text{zero} = 0$, and the Control Unit selects the branch target: $\text{PC} = 7$

In our verification program, we also tested the opposite case where the two registers contain equal values. In that case $zero = 1$, so the BNE instruction is not taken and the PC advances normally.

3.4 System Instructions

The “11” instruction class is reserved for system-level instructions. The project defines two instructions in this class:

1100... → HALT

1101... → NOP

HALT stops normal processor execution by freezing the PC, while **NOP** performs no register write and allows the PC to advance normally. Final_project_Mini_cpu_dd_camp.pdfPDF

This is implemented through the pc_src control signal:

HALT → pc_src = 11 → Hold PC

NOP → pc_src = 00 → PC + 1

Thus, the ISA does not only define the operations performed by the ALU; it also defines how instructions control the overall flow of the processor.

4. Module Implementation

4.1 Program Counter (PC)

Purpose

The Program Counter (PC) stores the address of the current instruction being executed. In our Mini Processor, the PC is a 6-bit register, allowing it to address the 64 locations of the Instruction Memory.

Operation

The next PC value is selected according to the pc_src control signal:

- 00 → Increment the PC by 1 and execute the next sequential instruction.
- 01 → Load the jump target address.
- 10 → Load the branch target address when the branch condition is satisfied.

- 11 → Hold the current PC value. This is used by the HALT instruction.

The PC is updated synchronously on the **positive edge of the clock**. When reset is asserted, the PC is cleared to zero.

Signal	Direction	Width	Description
clk	Input	1	Clock signal
reset	Input	1	Synchronous active-high reset
pc_src	Input	2	Selects the next PC source
target	Input	6	Branch/jump target address
pc_out	Output	6	Current PC value

Verilog Implementation

```

1  module Program_Counter (clk,reset,pc_src,target,pc_out);
2
3      input clk,reset;
4      input [1:0] pc_src;
5      input [5:0] target;
6      output reg [5:0] pc_out;
7
8      always@(posedge clk)
9      begin
10         if (reset)
11             pc_out<=6'b0;
12         else
13             case(pc_src)
14                 2'b00: pc_out<=pc_out+1;
15                 2'b01: pc_out<=target;
16                 2'b10: pc_out<=target;
17                 2'b11: pc_out<=pc_out;
18             endcase
19         end
20     endmodule
21

```

Figure 4.1: Verilog implementation of the Program Counter.

4.2 Instruction Memory (IMEM)

Purpose

The Instruction Memory (IMEM) stores the processor's program instructions and provides the instruction corresponding to the current Program Counter (PC) address.

In our Mini Processor, the Instruction Memory is implemented as a **64 × 16-bit memory**, meaning it contains 64 instruction locations, with each location storing one 16-bit instruction.

Operation

The pc_out generated by the Program Counter is connected directly to the addr input of the Instruction Memory.

For every address, the corresponding 16-bit instruction is read from the memory and provided at the instr output. The memory is therefore used during the **instruction fetch stage** of the processor.

The instruction memory contents are loaded from an external .txt file using the \$readmemb system task. During verification, different memory files were used to load different test programs, allowing the processor instructions and control-flow operations to be tested independently.

The program itself is initialized from a memory file using \$readmemb.

Interface

Signal	Direction	Width	Description
addr	Input	6	Address from the Program Counter
instr	Output	16	Fetch instruction

Verilog Implementation

Figure 4.2: Verilog implementation of the Instruction Memory.

```
1  module Instruction_Memory(addr,instr);
2
3  input [5:0] addr;
4  output reg [15:0] instr;
5
6  reg [15:0] mem[63:0];
7
8  always @(*)
9      instr = mem[addr] ;
10
11
12  initial
13  begin
14      $readmemb("program_mem.txt", mem);
15  end
16
17  endmodule
```

Integration

The Instruction Memory receives the current pc_out from the Program Counter and returns the corresponding instruction. The fetched instruction is then sent to the Control Unit and the rest of the processor datapath for decoding and execution.

4.3 Register File

Purpose

The Register File provides the processor with a small set of fast internal storage locations used to hold operands and intermediate results.

In our Mini Processor, the Register File contains **four registers**, each with a width of **8 bits**.

Therefore, the registers are addressed using **2-bit addresses**.

Operation

The Register File supports:

- **Two combinational read ports**, allowing two register values to be read simultaneously.
- **One synchronous write port**, allowing one register to be updated on the rising edge of the clock.
- **Synchronous active-high reset**, which clears all four registers to zero.

The register addresses are extracted from the instruction by the top-level processor according to the instruction type.

For R-Type instructions:

- rs1 selects the first source register.
- rs2 selects the second source register.
- rd selects the destination register.

For I-Type instructions, the destination register is selected from the corresponding instruction field, while the immediate value is used as the second operand/write-back value.

Interface

Signal	Direction	Width	Description
clk	Input	1	Clock signal
rst	Input	1	Synchronous active-high reset
wr_en	Input	1	Enables register write
wr_addr	Input	2	Destination register address

Signal	Direction	Width	Description
wr_data	Input	8	Data to be written
rd_addr1	Input	2	First read address
rd_addr2	Input	2	Second read address
rd_data1	Output	8	First register data
rd_data2	Output	8	Second register data

Verilog Implementation

Figure 4.3: Verilog implementation of the Register File.

```

1  module Register_File(
2      input clk,rst,wr_en,
3      input [1:0] wr_addr, //rd
4      input [7:0] wr_data,
5      input [1:0] rd_addr1, //rs1
6      input [1:0] rd_addr2, //rs2
7      output [7:0] rd_data1, // value of rs1
8      output [7:0] rd_data2 // value of rs2
9  );
10     reg [7:0] registers [3:0];
11     integer i;
12     assign rd_data1 = registers[rd_addr1];
13     assign rd_data2 = registers[rd_addr2];
14     always @ (posedge clk)
15     begin
16         if(rst)
17         begin
18             for(i=0;i<4;i=i+1)
19                 registers[i]<=8'b0;
20             end
21         else if (wr_en)
22             registers[wr_addr] <= wr_data;
23         end
24     end
25 endmodule
26
27

```

Integration

The two outputs of the Register File are connected to the a and b inputs of the ALU. The ALU result, or the immediate value for I-Type instructions, is selected as the write-back data and returned to the Register File when wr_en is asserted.

4.4 Arithmetic Logic Unit (ALU)

Purpose

The Arithmetic Logic Unit (ALU) is responsible for performing the arithmetic, logical, and bit-manipulation operations required by the processor.

In our Mini Processor, the ALU operates on two **8-bit operands** and uses a **4-bit opcode** to select the required operation.

Operation

The ALU supports the operations defined in the project ISA, including:

- Addition and subtraction
- Division and multiplication
- Shift and rotate operations
- Increment and complement operations
- AND, NAND, NOR, OR
- XOR and XNOR

The serial_in input is used by the shift operation.

The ALU also generates a **zero flag**. The zero output is set to 1 whenever the ALU result is equal to zero; otherwise, it is set to 0.

This zero flag is particularly important for the **BEQ and BNE instructions**, where the processor uses the result of the subtraction between the two compared registers to determine whether the branch condition is satisfied.

Interface

Signal	Direction	Width	Description
a	Input	8	First operand
b	Input	8	Second operand
opcode	Input	4	Selects the ALU operation
serial_in	Input	1	Serial input for shift operation

MINI PROCESSOR — SINGLE-CYCLE CPU

Digital IC Design Camp | Final Project

Signal	Direction	Width	Description
result	Output	8	ALU operation result
zero	Output	1	Indicates whether the result is zero

Verilog Implementation

```
1  module ALU(a,b,opcode,serial_in,result,zero);
2
3  input [7:0] a,b;
4  input [3:0] opcode;
5  input serial_in;
6
7  output reg [7:0] result;
8  output reg zero;
9
10 always@(*)
11 begin
12     case(opcode)
13     4'b0000: result = a + b;
14     4'b0001: result = a - b;
15     4'b0010:
16     if (b!=0)
17     result = a / b;
18     else
19     result = 8'b0;
20     4'b0011: result = a * b;
21     4'b0100: result = {a[6:0], serial_in};
22     4'b0101: result = {a[6:0], a[7]};
23     4'b0110: result = ~a;
24     4'b0111: result = a + 1;
25     4'b1001: result = ~b;
26     4'b1010: result = a & b;
27     4'b1011: result = ~(a & b);
28     4'b1100: result = ~(a | b);
29     4'b1101: result = a | b;
30     4'b1110: result = a ^ b;
31     4'b1111: result = ~(a ^ b);
32     default: result = 8'b0;
33     endcase
34 end
35
36 if (result==8'b0)
37     zero=1;
38 else
39     zero=0;
40
41 end
42 endmodule
43
44
```

Figure 4.4: Verilog implementation of the Arithmetic Logic Unit (ALU).

Integration

The two operands are provided by the Register File, while the opcode is generated by the Control Unit. The resulting ALU output is either written back to the Register File or used to generate the zero flag for branch decisions.

4.5 Control Unit

Purpose

The Control Unit decodes the 16-bit instruction and generates the control signals required to execute it.

Operation

The Control Unit determines the instruction type using `instr[15:14]` and generates:

- `reg_write` — enables register write-back.
- `alu_op` — selects the ALU operation.
- `pc_src` — determines the next PC source.
- `wb_src` — selects between the ALU result and the immediate value.

For branch instructions, the zero flag from the ALU is used to determine whether the branch should be taken. The Control Unit also distinguishes between **BEQ** and **BNE** according to the implemented instruction encoding.

Interface

Signal	Direction	Width	Description
<code>instr</code>	Input	16	Current instruction
<code>zero</code>	Input	1	ALU zero flag
<code>reg_write</code>	Output	1	Register write enable
<code>alu_op</code>	Output	4	ALU operation
<code>pc_src</code>	Output	2	PC source selection
<code>wb_src</code>	Output	1	Write-back source selection

Verilog Implementation

```

1  module Control_Unit(
2      input [15:0]instr,
3      input zero,
4      output reg reg_write , wb_src,
5      output reg [3:0] alu_op,
6      output reg [1:0] pc_src
7  );
8      wire [1:0] decision;
9      assign decision = instr[15:14];
10     always @ (*)
11     begin
12         // default values
13         pc_src=2'b11;
14         wb_src=0;
15         alu_op=4'b1000;
16         reg_write=0;
17         case (decision)
18             2'b00:
19                 begin
20                     alu_op=instr[13:10];
21                     reg_write=1;
22                     pc_src=2'b0;
23                     wb_src=0;
24                     end
25
26             2'b01:
27                 begin
28                     alu_op=4'b1000;
29                     reg_write=1;
30                     pc_src=2'b0;
31                     wb_src=1;
32                     end
33
34             2'b10:
35                 begin
36                     alu_op=4'b0001;
37                     reg_write=0;
38                     wb_src=0;
39                     if (instr[13]==1'b0)
40                         pc_src=2'b01;
41                     else if (instr[13]==1'b1 && instr[8]==1'b0 && zero==1)
42                         pc_src=2'b10;
43                     else if (instr[13]==1'b1 && instr[8]==1'b0 && zero==0)
44                         pc_src=2'b00;
45                     else if (instr[13]==1'b1 && instr[8]==1'b1 && zero==0)
46                         pc_src=2'b10;
47                     else if (instr[13]==1'b1 && instr[8]==1'b1 && zero==1)
48                         pc_src=2'b00;
49                     end
50
51             2'b11:
52                 begin
53                     alu_op=4'b1000;
54                     reg_write=0;
55                     wb_src=0;
56                     if (instr[12]==1'b0)
57                         pc_src=2'b11;
58                     else if (instr[12]==1'b1)
59                         pc_src=2'b00;
60                     end
61
62         endcase
63     end
64 endmodule

```

Figure 4.5: Verilog implementation of the Control Unit.

Integration

The generated control signals are distributed to the PC, Register File, ALU, and write-back path to control the execution of each instruction.

4.6 Top-Level Mini Processor

Purpose

The Top-Level module connects all processor components together to form the complete **single-cycle Mini Processor**.

Operation

The module connects the main data path and control signals between:

- Program Counter
- Instruction Memory
- Register File
- ALU
- Control Unit

The current instruction is fetched from the Instruction Memory using the PC value. Its fields are then used to select the required registers, ALU operation, immediate value, and control signals.

The ALU result or immediate value is selected for write-back according to `wb_src`, while `pc_src` determines the next PC value.

Verilog Implementation

```

1  module mini_processor(
2      input clk,reset,serial_in,
3      output [5:0] pc_out,
4      output [15:0] instr,
5      output zero,
6      output reg_write
7  );
8
9      wire [15:0] instr_wire=instr[15:0];
10     wire zero_wire;
11     wire [5:0] pc_out_wire;
12     wire [3:0] instr_op_wire;
13     wire [1:0] rd_addr1_wire= (instr[15:14]==2'b00)? instr[7:6] : instr[12:11] ;
14     wire [1:0] rd_addr2_wire= (instr[15:14]==2'b00)? instr[5:4] : instr[10:9] ;
15     wire [1:0] wr_addr_wire= (instr[15:14]==2'b00)? instr[9:8] : instr[13:12] ;
16     wire [1:0] pc_src_wire;
17     wire [5:0] instr_target_wire = instr[5:0];
18     wire [7:0] rd_data1_a ;
19     wire [7:0] rd_data2_b;
20     wire [7:0] immediate = instr[11:4];
21     wire [7:0] alu_result_wire;
22     wire wb_src_wire;
23     wire [7:0] write_back_wire;
24
25     assign pc_out = pc_out_wire;
26     assign zero = zero_wire;
27     assign write_back_wire = (wb_src_wire == 0) ? alu_result_wire : immediate;
28
29     Program_Counter pc_inst(.clk(clk), .reset(reset) , .pc_out(pc_out_wire) , .target(instr_target_wire) ,
30     .pc_src(pc_src_wire) );
31
32     Instruction_Memory imem_inst( .addr(pc_out_wire) , .instr(instr) );
33
34     Register_File rg_inst(.clk(clk), .rst(reset) , .wr_data(write_back_wire) , .wr_addr(wr_addr_wire) ,
35     .wr_en(reg_write) , .rd_addr1(rd_addr1_wire) , .rd_addr2(rd_addr2_wire), .rd_data1(rd_data1_a),
36     .rd_data2(rd_data2_b) );
37
38     ALU alu_inst(.a(rd_data1_a) , .b(rd_data2_b) , .opcode(instr_op_wire) , .serial_in(serial_in) ,
39     .zero(zero_wire) , .result(alu_result_wire) );
40
41     Control_Unit cu_inst( .instr(instr_wire) , .zero(zero_wire) , .reg_write(reg_write) ,
42     .pc_src(pc_src_wire) , .alu_op(instr_op_wire) , .wb_src(wb_src_wire) );
43
44
45
46
47
48
49
50     endmodule

```

Figure 4.6: Verilog implementation of the Top-Level Mini Processor.

Integration

This module represents the complete connection of the processor components and provides the required external inputs and outputs for the final system.

4.7 Testbench

A dedicated Verilog testbench was used to verify the complete Mini Processor. The testbench generates the clock and reset signals, provides the serial_in input, and instantiates the mini_processor as the Unit Under Test (UUT).

During simulation, the testbench was used to observe the processor outputs and internal signals, including the **PC, instruction, register write enable, and ALU zero flag**. Different instruction memory files were then used to test the required processor operations and control-flow instructions.

```
1  `timescale 1ns/1ps
2  module MINI_PROCESSOR_tb;
3
4  reg clk,reset,serial_in;
5  wire [5:0] pc_out;
6  wire [15:0] instr;
7  wire zero,reg_write;
8
9  mini_processor uut (
10     .clk(clk) , .reset(reset),
11     .serial_in(serial_in), .pc_out(pc_out),
12     .instr(instr), .zero(zero) ,
13     .reg_write(reg_write)
14 );
15 always #100 clk = ~clk;
16
17 initial
18     begin
19         clk=0;
20         reset=1;
21         serial_in=0;
22         @(negedge clk) reset=0;
23
24         #250 reset=1;
25         #60 reset=0;
26
27         #2000;
28         $display("HALT CHECK: PC = %d", pc_out);
29         #100;
30         $display("HALT CHECK: PC = %d", pc_out);
31
32         $finish;
33     end
34
35 initial
36     begin
37         $monitor("Time=%0t | Reset=%b | PC=%d | Instr=%b | Zero=%b | RegWrite=%b | R0=%d R1=%d R2=%d R3=%d",
38             $time, reset, pc_out, instr, zero, reg_write,
39             uut.rg_inst.registers[0],
40             uut.rg_inst.registers[1],
41             uut.rg_inst.registers[2],
42             uut.rg_inst.registers[3]);
43     end
44
45 endmodule
46
```

Figure 4.7: Verilog testbench used to simulate the Mini Processor.

5. Functional Requirements

The implemented Mini Processor satisfies the required processor functions through the integrated datapath and control logic.

5.1 Instruction Execution

The processor successfully supports the implemented instruction types:

- **R-Type:** ALU operations using two register operands.
- **I-Type:** Immediate operations using an 8-bit immediate value.
- **Branch/Jump:** BEQ, BNE, and JMP.
- **System:** HALT and NOP.

5.2 Program Counter Control

The PC supports:

- Sequential execution using $PC + 1$.
- Jump and branch target loading.
- PC hold during HALT.

5.3 Register and ALU Operations

The Register File supports two simultaneous reads and one synchronous write. The ALU performs the required 16 operations and generates the zero flag used by BEQ and BNE.

5.4 Verification

The processor was tested using different memory programs to verify:

- Normal instruction execution.
- Taken and not-taken BEQ.
- Taken and not-taken BNE.
- JMP.
- HALT PC freeze.

6. Verification

The Mini Processor was verified through simulation using different instruction memory files. Each test program was stored in a separate .txt file and loaded into the Instruction Memory using the \$readmemb system task. The test programs were designed to verify arithmetic and logical operations, immediate instructions, conditional branches, jumps, and the HALT operation.

6.1 Main Instruction Test

The first test was performed using the program_mem.txt file. This program was designed to verify the basic instruction execution and the main control-flow operations of the processor.

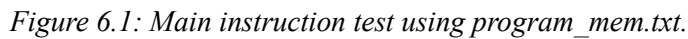
The contents of the file are:

```
0100000001010000 // LOADI R0,5
0101000000110000 // LOADI R1,3
0000001000010000 // ADD R2,R0,R1
0000011100010000 // SUB R3,R0,R1
1011011000000110 // BEQ R2,R3,6
0011101100010000 // XOR R3,R0,R1
1000000000000111 // JMP 7
1100000000000000 // HALT
```

The instructions are executed sequentially as follows:

- LOADI R0,5 loads the immediate value **5** into R0.
- LOADI R1,3 loads the immediate value **3** into R1.
- ADD R2,R0,R1 adds R0 and R1, so R2 = 8.
- SUB R3,R0,R1 subtracts R1 from R0, so R3 = 2.
- BEQ R2,R3,6 compares R2 and R3. Since $8 \neq 2$, the branch is **not taken**, and the PC continues to the next instruction.
- XOR R3,R0,R1 performs the XOR operation between R0 and R1 and stores the result in R3.
- JMP 7 changes the PC directly to address **7**.

- This test therefore verifies the correct operation of **immediate loading, ALU arithmetic, conditional branching, logical operation, jump, and HALT.**



```
# Time=0 | Reset=1 | PC= x | Instr=xxxxxxxxxxxxxxxxxx | Zero=1 | RegWrite=0 | R0= x R1= x R2= x R3= x
# Time=100000 | Reset=1 | PC= 0 | Instr=0100000001010000 | Zero=1 | RegWrite=1 | R0= 0 R1= 0 R2= 0 R3= 0
# Time=200000 | Reset=0 | PC= 0 | Instr=0100000001010000 | Zero=1 | RegWrite=1 | R0= 0 R1= 0 R2= 0 R3= 0
# Time=300000 | Reset=0 | PC= 1 | Instr=0101000000110000 | Zero=1 | RegWrite=1 | R0= 5 R1= 0 R2= 0 R3= 0
# Time=450000 | Reset=1 | PC= 1 | Instr=0101000000110000 | Zero=1 | RegWrite=1 | R0= 5 R1= 0 R2= 0 R3= 0
# Time=500000 | Reset=1 | PC= 0 | Instr=0100000001010000 | Zero=1 | RegWrite=1 | R0= 0 R1= 0 R2= 0 R3= 0
# Time=510000 | Reset=0 | PC= 0 | Instr=0100000001010000 | Zero=1 | RegWrite=1 | R0= 0 R1= 0 R2= 0 R3= 0
# Time=700000 | Reset=0 | PC= 1 | Instr=0101000000110000 | Zero=1 | RegWrite=1 | R0= 5 R1= 0 R2= 0 R3= 0
# Time=900000 | Reset=0 | PC= 2 | Instr=0000001000010000 | Zero=0 | RegWrite=1 | R0= 5 R1= 3 R2= 0 R3= 0
# Time=1100000 | Reset=0 | PC= 3 | Instr=0000011100010000 | Zero=0 | RegWrite=1 | R0= 5 R1= 3 R2= 8 R3= 0
# Time=1300000 | Reset=0 | PC= 4 | Instr=1011011000000110 | Zero=0 | RegWrite=0 | R0= 5 R1= 3 R2= 8 R3= 2
# Time=1500000 | Reset=0 | PC= 5 | Instr=0011101100010000 | Zero=0 | RegWrite=1 | R0= 5 R1= 3 R2= 8 R3= 2
# Time=1700000 | Reset=0 | PC= 6 | Instr=1000000000000111 | Zero=1 | RegWrite=0 | R0= 5 R1= 3 R2= 8 R3= 6
# Time=1900000 | Reset=0 | PC= 7 | Instr=1100000000000000 | Zero=1 | RegWrite=0 | R0= 5 R1= 3 R2= 8 R3= 6
# HALT CHECK: PC = 7
# HALT CHECK: PC = 7
```

Figure 6.2: Console output of the main instruction test.

The second test was performed using `program_beq_taken.txt`. This program was specifically created to verify the case where the BEQ condition is satisfied.

The contents of the file are:

```
0100000001010000 // LOADI R0,5
0101000000110000 // LOADI R1,3
0111000001100000 // LOADI R3,6
0110000001100000 // LOADI R2,6
1011011000000110 // BEQ R2,R3,6
1100000000000000 // HALT
1100000000000000 // HALT
```

The program first loads R3 and R2 with the same value:

- `LOADI R3,6` → `R3 = 6`
- `LOADI R2,6` → `R2 = 6`

Then the instruction:

`BEQ R2,R3,6`

compares the two registers using the ALU subtraction operation. Since:

$$R2 - R3 = 6 - 6 = 0$$

the ALU generates `zero = 1`, indicating that the two registers are equal. Therefore, the branch is taken and the PC changes from **address 4 to address 6**, skipping the `HALT` instruction at address 5 and executing the instruction at the branch target.

This test confirms that the processor correctly detects a zero ALU result and uses it to take a `BEQ` branch.

MINI PROCESSOR — SINGLE-CYCLE CPU

Digital IC Design Camp | Final Project

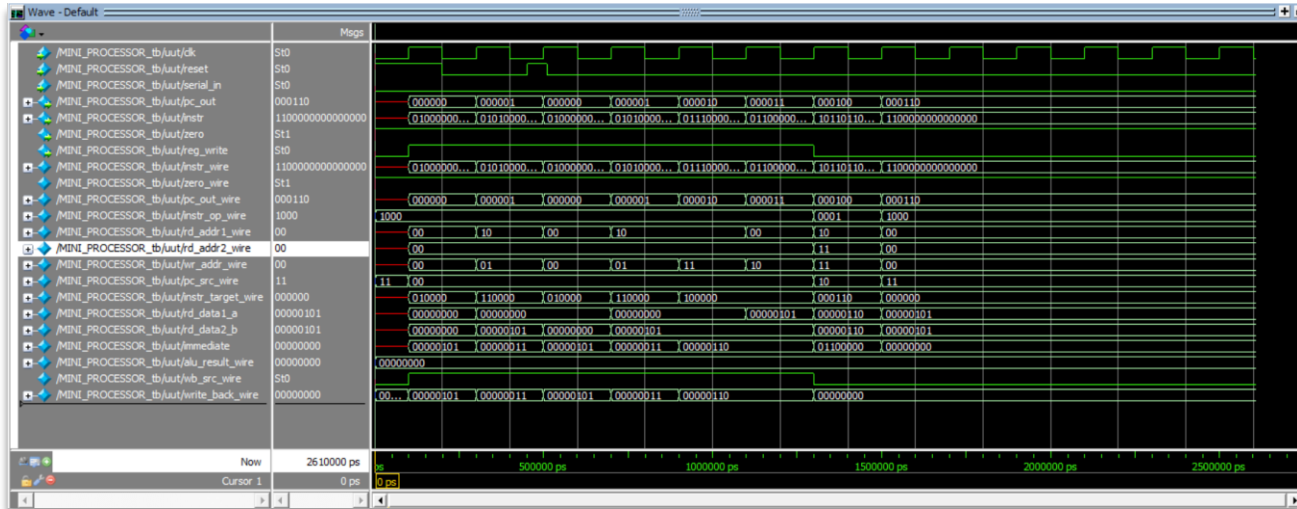


Figure 6.3: Taken BEQ test using program_beq_taken.txt.

The following console output confirms the simulation results and shows the final values obtained during the test.

```
# Time=0 | Reset=1 | PC= x | Instr=xxxxxxxxxxxxxxxx | Zero=1 | RegWrite=0 | R0= x R1= x R2= x R3= x
# Time=100000 | Reset=1 | PC= 0 | Instr=0100000001010000 | Zero=1 | RegWrite=1 | R0= 0 R1= 0 R2= 0 R3= 0
# Time=200000 | Reset=0 | PC= 0 | Instr=0100000001010000 | Zero=1 | RegWrite=1 | R0= 0 R1= 0 R2= 0 R3= 0
# Time=300000 | Reset=0 | PC= 1 | Instr=0101000000110000 | Zero=1 | RegWrite=1 | R0= 5 R1= 0 R2= 0 R3= 0
# Time=450000 | Reset=1 | PC= 1 | Instr=0101000000110000 | Zero=1 | RegWrite=1 | R0= 5 R1= 0 R2= 0 R3= 0
# Time=500000 | Reset=1 | PC= 0 | Instr=01000000001010000 | Zero=1 | RegWrite=1 | R0= 0 R1= 0 R2= 0 R3= 0
# Time=510000 | Reset=0 | PC= 0 | Instr=0100000001010000 | Zero=1 | RegWrite=1 | R0= 0 R1= 0 R2= 0 R3= 0
# Time=700000 | Reset=0 | PC= 1 | Instr=0101000000110000 | Zero=1 | RegWrite=1 | R0= 5 R1= 0 R2= 0 R3= 0
# Time=900000 | Reset=0 | PC= 2 | Instr=0111000001100000 | Zero=1 | RegWrite=1 | R0= 5 R1= 3 R2= 0 R3= 0
# Time=1100000 | Reset=0 | PC= 3 | Instr=0110000001100000 | Zero=1 | RegWrite=1 | R0= 5 R1= 3 R2= 0 R3= 6
# Time=1300000 | Reset=0 | PC= 4 | Instr=10110110000000110 | Zero=1 | RegWrite=0 | R0= 5 R1= 3 R2= 6 R3= 6
# Time=1500000 | Reset=0 | PC= 6 | Instr=1100000000000000 | Zero=1 | RegWrite=0 | R0= 5 R1= 3 R2= 6 R3= 6
# HALT CHECK: PC = 6
# HALT CHECK: PC = 6
```

Figure 6.4: Console output of the BEQ instruction test.

6.3 BNE Test

The bonus BNE instruction was verified using the program_bne.txt file. The program contains two BNE instructions to verify both possible cases: **branch not taken** and **branch taken**.

The contents of the file are:

```
0100000001010000 // LOADI R0,5
0101000001110000 // LOADI R1,7
0110000001010000 // LOADI R2,5
```

MINI PROCESSOR — SINGLE-CYCLE CPU

Digital IC Design Camp | Final Project

```
0111000001010000 // LOADI R3,5
1011011100000110 // BNE R2,R3,6
1010110100000111 // BNE R1,R2,7
1100000000000000 // HALT
1100000000000000 // HALT
```

The first four instructions initialize the registers:

- R0 = 5
- R1 = 7
- R2 = 5
- R3 = 5

The first branch:

BNE R2,R3,6

is **not taken** because $R2 = R3 = 5$. The subtraction produces zero, so the PC continues normally to address 5.

The second branch:

BNE R1,R2,7

is **taken** because $R1 = 7$ while $R2 = 5$. The subtraction produces a non-zero result, so $\text{zero} = 0$, and the PC changes from **address 5 to address 7**.

This test confirms that the implemented BNE instruction correctly branches when the two register values are different and does not branch when they are equal.

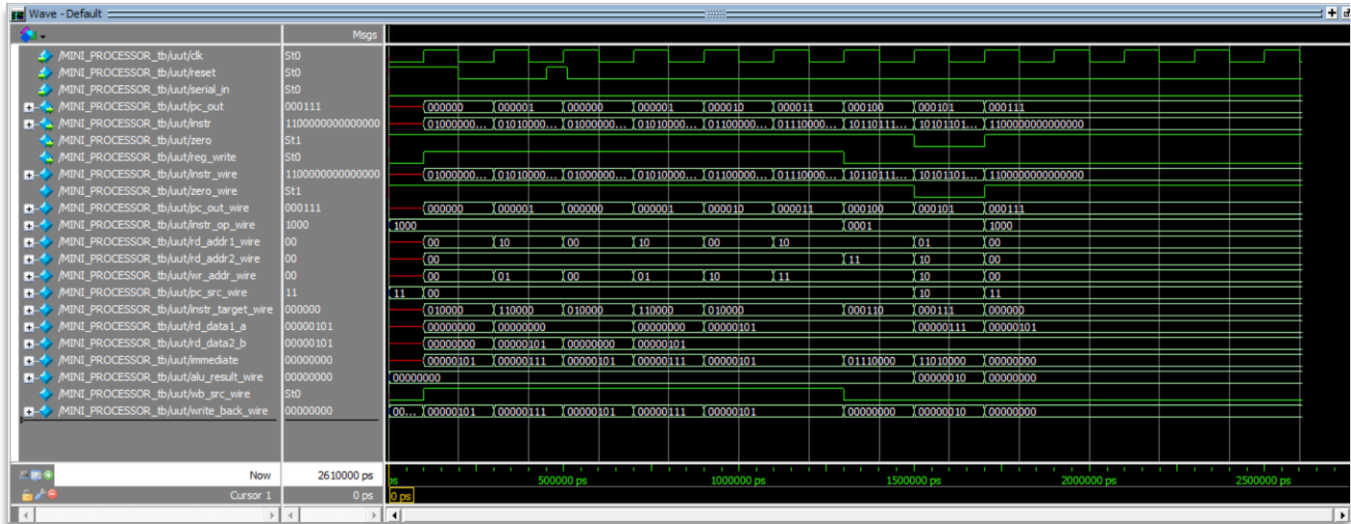


Figure 6.5: BNE test using program_bne.txt.

The following console output confirms the simulation results and shows the final values obtained during the test.

Figure 6.6: Console output of the BNE instruction test.

```
# Time=0 | Reset=1 | PC= x | Instr=xxxxxxxxxxxxxxxx | Zero=1 | RegWrite=0 | R0= x R1= x R2= x R3= x
# Time=100000 | Reset=1 | PC= 0 | Instr=0100000001010000 | Zero=1 | RegWrite=1 | R0= 0 R1= 0 R2= 0 R3= 0
# Time=200000 | Reset=0 | PC= 0 | Instr=0100000001010000 | Zero=1 | RegWrite=1 | R0= 0 R1= 0 R2= 0 R3= 0
# Time=300000 | Reset=0 | PC= 1 | Instr=0101000001110000 | Zero=1 | RegWrite=1 | R0= 5 R1= 0 R2= 0 R3= 0
# Time=450000 | Reset=1 | PC= 1 | Instr=0101000001110000 | Zero=1 | RegWrite=1 | R0= 5 R1= 0 R2= 0 R3= 0
# Time=500000 | Reset=1 | PC= 0 | Instr=0100000001010000 | Zero=1 | RegWrite=1 | R0= 0 R1= 0 R2= 0 R3= 0
# Time=510000 | Reset=0 | PC= 0 | Instr=0100000001010000 | Zero=1 | RegWrite=1 | R0= 0 R1= 0 R2= 0 R3= 0
# Time=700000 | Reset=0 | PC= 1 | Instr=0101000001110000 | Zero=1 | RegWrite=1 | R0= 5 R1= 0 R2= 0 R3= 0
# Time=900000 | Reset=0 | PC= 2 | Instr=0110000001010000 | Zero=1 | RegWrite=1 | R0= 5 R1= 7 R2= 0 R3= 0
# Time=1100000 | Reset=0 | PC= 3 | Instr=0111000001010000 | Zero=1 | RegWrite=1 | R0= 5 R1= 7 R2= 5 R3= 0
# Time=1300000 | Reset=0 | PC= 4 | Instr=1011011100000110 | Zero=1 | RegWrite=0 | R0= 5 R1= 7 R2= 5 R3= 5
# Time=1500000 | Reset=0 | PC= 5 | Instr=1010110100000111 | Zero=0 | RegWrite=0 | R0= 5 R1= 7 R2= 5 R3= 5
# Time=1700000 | Reset=0 | PC= 7 | Instr=1100000000000000 | Zero=1 | RegWrite=0 | R0= 5 R1= 7 R2= 5 R3= 5
# HALT CHECK: PC = 7
# HALT CHECK: PC = 7
```

6.4 HALT Verification

The HALT instruction was verified as part of the previous test programs. When the processor reaches a HALT instruction, the Control Unit sets pc_src = 11, which causes the Program Counter to hold its current value.

During simulation, the PC remained unchanged after reaching HALT, confirming that instruction execution stops correctly and the processor does not continue to the following memory locations.

```
# Time=1700000 | Reset=0 | PC= 7 | Instr=1100000000000000 | Zero=1 | RegWrite=0 | R0= 5 R1= 7 R2= 5 R3= 5
# HALT CHECK: PC = 7
# HALT CHECK: PC = 7
```

Figure 6.7: PC hold operation during HALT.

7. Future Improvements

The current Mini Processor provides a functional single-cycle CPU with the required instruction set and verified control-flow operations. However, the design can be further developed by adding more instructions and extending the processor capabilities.

Future improvements could include expanding the register file, adding new instruction types, improving the memory system, and introducing more advanced processor features such as pipelining.

This project provides a solid foundation that can be extended into a more capable processor design in future work.